

Protocol Vision V3

Pre-Standardization Core v0.1 and Codex Build Directive

North Star: Upgrade the naming layer into NBSR. NBSR resolves every name. Legacy destinations continue through DNS-compatible resolution; NBSR-enabled destinations resolve into authenticated secure routes without revealing the origin address to the client.

AUTHORITATIVE DIRECTION

Status	Authoritative architecture direction and implementation program
Version	3.0 working draft
Date	2026-07-26
Audience	Protocol implementers, Codex, security reviewers, network operators, clouds, ISPs, and standards contributors
Primary decision	NBSR is the universal resolver; its Source Edge acts for legacy devices in an upgraded network

This document is intentionally honest about the boundary between the hardened prototype, the first implementable protocol core, and the later work required for global federation.

Document map

Part	Purpose
Direction	Sections 1-3: precedence, North Star, and chosen resolver-first architecture
Architecture	Sections 4-9: components, terminology, naming, compatibility, handles, and records
Protocol core	Sections 10-18: wire profile, state machines, grants, security, lifecycle, federation, ISP, privacy, and failure
Deployment and evidence	Sections 19-21: plug-and-play upgrade, rollback, prototype baseline, and phased roadmap
Codex program	Sections 22-27: execution rules, work packages, conformance, errors, ready directive, and decision gates
Appendices	Normative language, standards baseline, and final anti-drift checklist

1. Document status and precedence

This document defines the intended direction of Name-Based Secure Routing. It converts the project from a collection of prototypes and architectural ideas into a bounded protocol-development program.

The precedence order is:

Document or implementation	Role after V3
NBSR Protocol Vision V3	Authoritative architecture, adoption model, protocol-core direction, invariants, and Codex work program
NBSR Protocol Vision V2	Superseded where it makes a per-device native client the primary adoption path; retained for security, lifecycle, ISP, federation, and anti-drift principles
Name-Based Secure Routing feasibility study	Supporting research on standards, open-source components, enterprise deployment, and operations
Hardened Build Week prototype	Verified proof of name-first routing, synthetic handles, signed bindings, opaque relay, hidden origin, enterprise authorization, and network isolation at prototype scale
Future NBSR Internet-Draft and conformance specification	Eventual normative public standard after independent implementations interoperate

V3 deliberately resolves one ambiguity:

NBSR is not a component that runs after DNS. An upgraded NBSR Name Node performs the name-resolution role itself. It preserves DNS-compatible behavior for legacy names and produces NBSR route state for NBSR-enabled names.

For legacy global names during migration, an NBSR Name Node may implement conventional recursion, host imported zones, or forward to legacy DNS infrastructure. To the client, NBSR is still the resolver. For an NBSR-enabled name, conventional DNS is not used to decide the secure route.

2. Executive direction

NBSR is a DNS-compatible name-resolution and secure-routing protocol. It has two mandatory architectural planes:

1. **Name/Resolution Plane** - accepts name queries, serves existing DNS records, evaluates NBSR service records, creates route intent, and returns a compatibility handle when an application still expects an IP address.
2. **Secure Route/Tunnel Plane** - converts the route intent into an authenticated encrypted path through a source edge and destination edge to a private origin connector.

These planes may be installed as one product or appliance, but they must remain separate processes and failure domains. A tunnel bug must not take down legacy name resolution. A resolver failure must not immediately terminate already-authorized active streams.

The Internet below NBSR remains recognizable:

- IP and BGP continue to move packets between NBSR edges.
- Existing applications continue to use sockets.
- TCP, UDP, HTTP, HTTPS, SSH, database protocols, and future protocols can be carried through NBSR.
- Public DNS remains available to networks and domains that have not migrated.
- A destination may operate in legacy, dual-published, or NBSR-secure-only mode.

NBSR changes the security and routing decision from:

```
name -> origin IP -> direct connection
```

to:

```
name -> NBSR resolution -> route intent/compatibility handle
-> source edge -> authenticated encrypted tunnel
-> destination edge -> outbound origin connector -> private service
```

The first implementation must prove this model without requiring an agent on a normal client behind an NBSR-enabled router, enterprise gateway, or ISP edge.

3. Chosen architecture and rejected alternatives

3.1 Chosen: resolver-first, dual-plane NBSR

The recommended architecture upgrades the network name service and places an NBSR source gateway on the traffic path. The source gateway acts as the NBSR protocol client on behalf of legacy endpoint devices.

Advantages:

- no per-application or per-device installation inside an upgraded network;
- gradual operator-by-operator adoption;
- legacy names continue to resolve normally;
- NBSR-enabled services can hide their origin;
- arbitrary legacy protocols can be associated with a name through a scoped synthetic handle;
- a future native `connect(name)` API can coexist with the same protocol core.

Required condition:

A standalone remote DNS server that cannot influence routing is insufficient. The operational "DNS upgrade" must include, or be paired with, a source gateway that receives traffic for the returned NBSR edge address or synthetic range.

3.2 Rejected as the primary path: client-first NBSR

A native client on every laptop, phone, and server provides the cleanest API and the strongest session mobility, but it is not a realistic first global adoption requirement. Native clients remain a later optimization for mobile networks, roaming devices, and direct `connect(name)` support.

3.3 Rejected: DNS as only a bootstrap hint

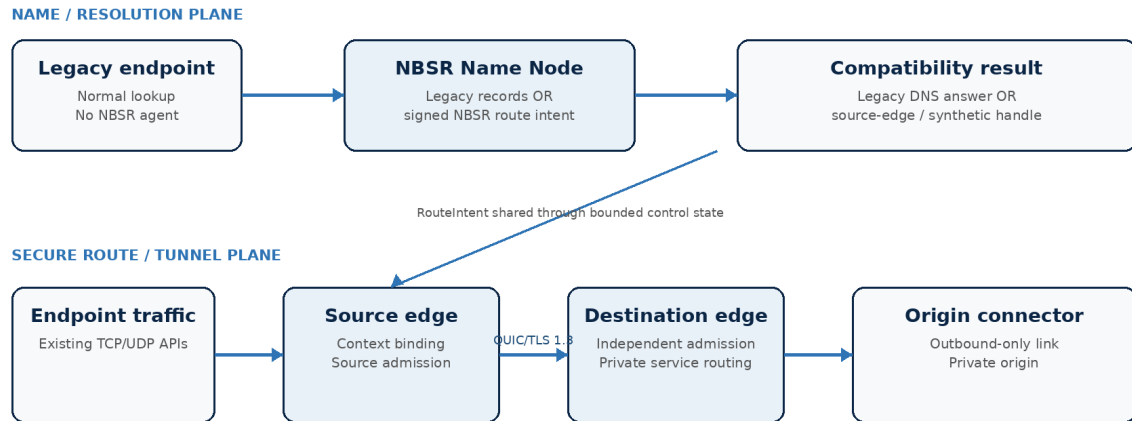
Using DNS only to point an aware client to a separate overlay preserves too much of the old architecture and requires endpoint software before adoption. It is useful as a compatibility mode, but it does not realize the "upgrade the name infrastructure" vision.

3.4 Rejected: put tunnels inside the DNS process

Name resolution and tunnel transport must be installable together but must not share one privileged process. The correct product shape is one deployable NBSR Node with separated services, credentials, capacity limits, and health checks.

4. Reference architecture

NBSR V3: one name service, two isolated planes



The product may be installed as one NBSR Node, but name resolution and tunnel transport remain separate processes, keys, resource limits, and failure domains.

Figure 1. NBSR V3 dual-plane resolver-first architecture

4.1 Logical components

Component	Required responsibility
Legacy endpoint	Uses normal name lookup and normal application sockets; no NBSR software is required when behind an NBSR source edge
NBSR Name Node	DNS-compatible resolver/authoritative service, NBSR registry lookup, route-intent creation, handle allocation, cache and fallback policy
Source NBSR Edge	Correlates a client/subscriber with a handle, authenticates route state, performs source admission, opens or reuses tunnels
Regional NBSR Control Plane	Selects destination operator/edge, distributes revocation and trust state, and coordinates route leases
Destination NBSR Edge	Independently validates admission, applies destination policy, and reaches the registered origin connector
Origin Connector	Creates outbound-only authenticated connectivity from a private service network to the destination edge
Registry/Federation Service	Publishes signed name ownership, delegation, service records, operator trust, sequence numbers, and revocation

4.2 Legacy endpoint flow

1. Endpoint asks its configured resolver for api.example.com.
2. NBSR Name Node resolves the name.
3. If the name is legacy, NBSR returns normal DNS-compatible data.
4. If the name is NBSR-enabled, NBSR creates RouteIntent state and returns:
 - a. a source-edge address; or
 - b. a scoped synthetic IP compatibility handle.
5. Endpoint opens its normal TCP/UDP connection.
6. Network routing sends the connection to the Source NBSR Edge.
7. Source Edge binds the flow to the original name and route context.
8. Source Edge opens or reuses an authenticated encrypted tunnel.
9. Destination Edge independently admits the route.
10. Origin Connector carries the flow to the private service.

4.3 Native client flow

Future native applications or operating systems may use:

```
connect(name, transport, service)
```

This removes synthetic addresses from the local compatibility path, but it must use the same name records, route grants, edges, tunnel profile, and conformance tests.

4.4 Knowledge separation

Participant	Permitted knowledge by default
Endpoint application	Requested name and local NBSR/compatibility handle; not the origin address
Source edge	Client/subscriber context, requested name, source policy, and opaque destination route
Federation transit	Operator and route identifiers required for forwarding; not client identity or origin address
Destination edge	Destination service and origin connector; not the endpoint's raw private identity unless policy explicitly requires it
Origin service	Destination connector/edge identity; not the direct client IP
Public Internet between edges	Encrypted NBSR transport between operator endpoints

This is traffic concealment and metadata minimization, not Tor-style anonymity. Source and destination operators may each know their local side, and colluding operators may correlate traffic.

5. Terminology

Term	Definition
Name	Hierarchical application-visible identifier requested through NBSR

Term	Definition
Legacy name	Name resolved to conventional DNS-compatible records without creating an NBSR route
NBSR-enabled name	Name with a signed NBSR service record that resolves to secure route state
Name Node	Resolver/authoritative component that handles both legacy and NBSR resolution
Resolution Context ID	Source-local identifier binding a name query to a client, subscriber, device, CPE, or authenticated resolver session
Synthetic handle	Scoped IP-shaped compatibility token representing NBSR route state; never an origin address
RouteIntent	Short-lived internal plan created by name resolution before the first matching connection
Route Grant	Signed, proof-of-possession-bound authorization for a source edge to request a specific route
Source edge	NBSR gateway closest to the requesting endpoint or subscriber
Destination edge	NBSR gateway authorized to reach a destination connector
Origin connector	Outbound-only agent or gateway that exposes a private service to its destination edge
Tunnel	Authenticated encrypted transport between NBSR protocol peers
Stream	One application flow multiplexed inside a tunnel
Lease	Bounded authorization interval for a route
Operator	ISP, cloud, enterprise, or other administrative domain running NBSR infrastructure
Federation	Signed trust, delegation, routing, and revocation relationship between operators

6. Naming and resolution semantics

6.1 One resolver, two outcomes

An upgraded NBSR Name Node **MUST** accept all configured name queries. It **MUST** classify the authoritative result as one of:

1. `LEGACY_RECORD_SET`
2. `NBSR_SERVICE_RECORD`
3. `NEGATIVE_OR_ERROR`

For `LEGACY_RECORD_SET`, the Name Node returns standards-compatible DNS data according to normal cache and delegation rules.

For `NBSR_SERVICE_RECORD`, the Name Node:

- validates ownership, signature, sequence, validity interval, and revocation;
- identifies the destination operator and acceptable route profiles;
- creates `RouteIntent` state bound to a Resolution Context ID;
- returns a source-edge address or synthetic compatibility handle;

- does not place the origin address in the client response, client-visible cache, or client route state.

For **NEGATIVE_OR_ERROR**, the Name Node returns a stable DNS-compatible or native NBSR error without silently bypassing security policy.

6.2 Resolution Context ID

The Name Node and Source Edge **MUST** share a non-exported Resolution Context ID. It can be derived from:

- a local router client lease;
- an ISP subscriber session;
- an authenticated DoH/DoT session;
- a CPE tunnel;
- a native NBSR client session;
- an enterprise device or workload identity extension.

Raw source IP alone is insufficient when NAT, shared resolvers, roaming, or proxying can merge clients. A deployment that cannot preserve a trustworthy resolution context **MUST NOT** use per-client synthetic mappings. It may use a shared web edge mode or require a local adapter/native client.

6.3 Name normalization

Protocol Core v0.1 uses:

- lowercase DNS A-label form;
- no trailing dot in canonical form;
- labels of 1-63 octets;
- total canonical name length not exceeding 253 octets;
- rejection of empty labels, embedded NUL, IP literals as names, and ambiguous Unicode;
- IDNA conversion at the presentation boundary, not inside signed canonical fields.

The canonical name is included, directly or by collision-resistant digest, in every route authorization.

6.4 Cache separation

NBSR maintains distinct caches:

Cache	Typical lifetime	Security meaning
Legacy DNS data	Record TTL	Conventional resolution
Signed NBSR service record	Record validity and cache TTL	Ownership and service metadata only
Synthetic-handle mapping	DNS answer TTL plus bounded connection grace	Compatibility correlation, not authorization
RouteIntent	Seconds	Prepared route decision
Route Grant	Seconds to minutes	Admission authorization

Cache	Typical lifetime	Security meaning
Active tunnel	Longer, with rotation	Encrypted transport
Revocation state	SLO-bound	Immediate or bounded invalidation

A DNS TTL MUST NOT be treated as an authorization lease.

7. Compatibility and migration contract

7.1 Destination publication modes

Mode	Upgraded NBSR network	Non-upgraded network	Security behavior
Legacy	Receives conventional DNS result	Receives conventional DNS result	No NBSR route
Dual-published	Uses NBSR route	Uses conventional public compatibility endpoint	Migration mode; public endpoint remains exposed
NBSR-preferred	Uses NBSR; explicit operator fallback may exist	Uses legacy endpoint	Fallback is an owner policy, never an automatic downgrade
NBSR-secure-only	Uses NBSR route	Cannot reach the service	Origin can remain non-public

There is an unavoidable trade-off:

A service cannot simultaneously remove every public compatibility endpoint and remain reachable by clients on networks that do not support NBSR. Global adoption requires a dual-published period or a public compatibility gateway.

7.2 No implicit downgrade

For an NBSR-enabled secure-only name:

- resolution or route failure MUST fail closed;
- the system MUST NOT query for or reveal an origin A/AAAA record as fallback;
- diagnostic bypass MUST be explicit, time-bounded, administrator-controlled, and auditable;
- a route profile downgrade MUST require signed owner/operator policy.

7.3 Existing applications

The compatibility path MUST preserve:

- original HTTPS hostname for SNI and certificate validation;
- opaque payload forwarding by default;
- TCP connection semantics;

- UDP datagram boundaries when UDP support is added;
- application-visible errors that do not reveal origin addressing;
- ordinary wake-up mechanisms such as APNs and FCM.

NBSR MUST NOT require TLS interception to route HTTPS.

8. Compatibility handle modes

8.1 Synthetic IP mode

Synthetic IP is the primary compatibility method for arbitrary legacy TCP and, later, UDP.

Properties:

- scoped to a Resolution Context ID and Source Edge;
- not globally routable as a destination service;
- routed only toward the local/ISP NBSR Source Edge;
- mapped to canonical name, allowed transport, service, and expiry;
- safe to reuse across unrelated source contexts when the gateway can distinguish them;
- retained at least through the advertised TTL plus bounded grace;
- never serialized as an origin location.

Production deployments MUST use operator-selected ranges that do not conflict with physical routes, VPNs, containers, Hyper-V, WSL, CGNAT, or other local overlays. The protocol MUST NOT assign one universal IPv4 synthetic prefix for the entire Internet.

IPv6 ULA or an operator-owned dedicated prefix is preferred when available. IPv4 ranges require explicit collision checks and routing policy.

8.2 Shared source-edge address mode

Multiple web names may resolve to a common NBSR source-edge address. This is suitable only when the source edge receives a trustworthy name signal, such as:

- an explicit native NBSR route token;
- unencrypted TLS SNI where ECH is not in use;
- HTTP Host in an allowed plaintext environment;
- an NBSR compatibility metadata channel.

Shared edge mode MUST NOT depend on payload inspection for arbitrary protocols. ECH, SSH, databases, opaque TCP, and most UDP protocols require synthetic or native name binding.

8.3 Native mode

Native clients receive a route object rather than an IP-shaped handle. Native mode is the long-term protocol API, but not the required first adoption mechanism.

9. NBSR service record

An NBSR Service Record is signed owner-controlled configuration. It **MUST NOT** contain session keys, reusable client grants, or public origin addresses.

Minimum logical fields:

```
record_version: 1
name: api.example.com
sequence: 42
owner_key_id: owner.example.com/2026-01
service_id: svc_4f6d...
destination_operator_id: op_example_net
destination_edge_set: edge-set-eu-1
origin_connector_id: oc_8a31...
transports:
- tcp
ports:
- 443
route_profiles:
- nbsr-quic-1
publication_mode: nbsr-secure-only
not_before: 1785000000
not_after: 1785086400
revocation_ref: revset_2026_207
signature: COSE_Sign1(...)
```

Required validation:

- owner/delegation chain is trusted;
- `sequence` is greater than the last accepted sequence for the same owner/name;
- validity interval is current;
- key and record are not revoked;
- destination operator is trusted for the requested profile;
- route capabilities do not exceed owner policy;
- rollback to an older signed record is rejected.

Single-operator Core v0.1 may use a local signed registry. Global federation is a later work package, but the record shape must not block it.

10. Protocol Core v0.1

10.1 Scope

Core v0.1 is a pre-standardization profile. It **MUST** implement:

- DNS-compatible legacy name service on the upgraded network;
- NBSR Service Record lookup in a local signed registry;
- synthetic handle allocation;
- source-edge correlation using Resolution Context ID;
- proof-of-possession route grants;
- source and destination admission;
- QUIC/TLS 1.3 encrypted tunnel between NBSR edges;
- multiplexed TCP streams;
- outbound origin connector;
- lease, renewal, drain, close, and revocation states;
- deterministic conformance tests.

Core v0.1 **MAY** defer:

- arbitrary UDP and QUIC application datagrams;
- live Wi-Fi-to-mobile migration;
- global federation;
- anycast;
- production HSM integration;
- independent second-language implementation.

10.2 Transport profile **nbsr-quic-1**

The first mandatory tunnel profile uses:

- QUIC version 1 semantics;
- TLS 1.3;
- ALPN protocol identifier **nbsr/1**;
- mutual NBSR edge authentication;
- forward secrecy;
- length-prefixed deterministic CBOR control messages;
- one bidirectional QUIC control stream;
- one QUIC stream per proxied TCP flow;
- explicit application-level admission before payload forwarding;
- key update according to TLS/QUIC limits and configured NBSR policy;
- no 0-RTT for route-changing control messages in Core v0.1.

UDP/443 is the default deployment port because it traverses existing networks more reliably than a newly allocated port. Operators may use another configured port; the wire protocol is selected by ALPN.

10.3 Serialization and signatures

Core v0.1 uses:

- deterministic CBOR for protocol structures;
- COSE Sign1 for signed service records, delegation statements, route grants, and revocation objects;
- Ed25519 as the required initial signature algorithm;
- SHA-256 for identifiers and thumbprints where a digest is required;
- X.509 or raw public-key trust only through a configured certified profile;
- JSON only for debugging, APIs, and human-readable fixtures, never as the only conformance encoding.

Algorithm agility **MUST** be versioned and downgrade-resistant. Implementations **MUST NOT** accept an algorithm merely because a library supports it.

10.4 Control message envelope

Every native control message has:

```
protocol_version
message_type
request_id
session_id
monotonic_sequence
body
```

Core v0.1 message types:

Code	Message
1	CLIENT_HELLO
2	EDGE_HELLO
3	ROUTE_OPEN
4	ROUTE_ACCEPT
5	ROUTE_REJECT
6	STREAM_OPEN
7	STREAM_ACCEPT
8	STREAM_REJECT

Code	Message
9	LEASE_RENEW
10	LEASE_RESULT
11	KEY_UPDATE_NOTICE
12	ROUTE_DRAIN
13	ROUTE_REVOKE
14	ROUTE_CLOSE
15	PING
16	PONG
17	ERROR

Numbers are frozen inside Core v0.1 test vectors. A later public draft may place them in an IANA-style registry.

11. Route and tunnel state machines

11.1 Resolution state

```
RECEIVED
-> NORMALIZED
-> LEGACY_RESOLVED
-> RESPONSE_SENT

RECEIVED
-> NORMALIZED
-> NBSR_RECORD_VALIDATED
-> ROUTE_INTENT_CREATED
-> HANDLE_BOUND
-> RESPONSE_SENT

Any validation failure
-> NEGATIVE_OR_ERROR
```

11.2 Tunnel state

```

IDLE
-> DISCOVERING
-> HANDSHAKING
-> AUTHENTICATING
-> ACTIVE
-> RENEWING
-> ACTIVE
-> DRAINING
-> CLOSED

```

Exceptional transitions:

- DISCOVERING | HANDSHAKING | AUTHENTICATING -> FAILED
- ACTIVE | RENEWING -> REVOKED
- ACTIVE -> MIGRATING -> ACTIVE in a later mobility profile
- ACTIVE -> FAILED -> RESUMING -> ACTIVE only when the route context remains valid

11.3 Stream state

```

NEW -> OPEN_PENDING -> OPEN -> HALF_CLOSED -> CLOSED
NEW | OPEN_PENDING -> REJECTED
OPEN -> RESET

```

An edge MUST NOT forward application payload until both source and destination admission are complete.

11.4 Origin connector state

```

DISCONNECTED
-> CONNECTING_OUTBOUND
-> AUTHENTICATED
-> READY
-> DRAINING
-> DISCONNECTED

```

The origin connector MUST initiate outbound connectivity. The protected origin SHOULD have no publicly reachable inbound service address.

12. Route Grant and proof of possession

A Route Grant is not a bearer ticket. It is signed and bound to a client/source session public key.

Required claims:

```

grant_version
route_id
canonical_name or name_digest
service_id
source_operator_id
source_edge_id
destination_operator_id
destination_edge_set
allowed_transports
allowed_ports or service capabilities
client_session_key_thumbprint
not_before
expires_at
lease_id
record_sequence
policy_hash
unique_nonce

```

For route opening, the source peer signs:

```

protocol_version || route_id || destination_edge_id ||
server_nonce || requested_transport || requested_service

```

The destination verifies:

- grant signature and issuer;
- grant time window;
- record sequence and revocation;
- proof-of-possession;
- destination edge membership;
- name/service/transport/port binding;
- nonce uniqueness;
- source operator trust;
- admission limits.

Replay protection MUST be partition-safe. A single-process cache is acceptable only in the local prototype profile.

13. Mandatory security invariants

An implementation MUST satisfy all of the following before claiming Core v0.1 conformance:

1. Applications or source gateways route by name, not origin IP.
2. Origin addressing never appears in an NBSR-enabled client response.
3. Every inter-edge native route is authenticated, encrypted, and integrity-protected.
4. Route authorization is proof-of-possession-bound and short-lived.

5. Names, capabilities, service, transport, and destination context are cryptographically bound.
6. Replay, downgrade, stale record, and rollback attempts fail closed.
7. Source and destination perform independent admission.
8. A compromised source edge cannot mint destination-authoritative grants.
9. A compromised destination edge cannot sign owner records.
10. Operator signing, tunnel identity, registry ownership, and deployment signing keys are separated.
11. Secure-only names never fall back automatically to raw origin resolution.
12. Active payload is not logged or inspected by the core protocol.
13. Resource use is bounded before expensive cryptographic or upstream work.
14. Control-plane unavailability does not expose the origin or downgrade the route.
15. Kubernetes, Envoy, OPA, SPIFFE, and any cloud platform remain optional implementations or extensions, not wire dependencies.

14. Lifecycle semantics

Core v0.1 distinguishes:

- **handle TTL** - compatibility cache lifetime;
- **route lease** - authorization lifetime;
- **idle timeout** - closes unused tunnel state;
- **maximum tunnel lifetime** - forces full re-establishment;
- **stream lifetime** - may outlive the admission grant within policy;
- **completion grace** - bounded time for an existing authorized stream to finish after new streams are denied.

Required behavior:

```
while authorized streams are active:
  renew before lease expiry
  revalidate revocation, ownership sequence, quotas, and policy
  rotate keys when required
  preserve stream authorization context
```

```
if renewal is denied:
  deny new streams
  apply immediate revoke or bounded completion grace
  close according to the signed revocation policy
```

Critical future acceptance test:

A large transfer continues through lease renewal, key rotation, source-edge failover, and a supported network-path migration without losing name or authorization binding.

15. Trust, registry, and federation direction

Global NBSR MUST NOT use one omnipotent root operator or a blockchain.

The preferred direction is:

- hierarchical or delegated name ownership;
- signed append-only transparency logs;
- operator federation;
- monotonic record sequence numbers;
- explicit delegation and revocation;
- independently auditable checkpoints;
- HSM/KMS-backed online operator keys;
- offline recovery/root keys;
- overlapping trust bundles during rotation;
- automatic isolation of compromised edges.

Federation must protect against:

- name hijacking;
- stale delegation;
- rollback;
- split-brain ownership;
- compromised operator key;
- malicious federation peer;
- inconsistent revocation views;
- network partitions.

Federation is not part of the first coding work package. Core data structures must, however, include owner, operator, sequence, validity, and revocation fields from the start.

16. ISP model and DDoS behavior

The ISP reference path is:

```
legacy endpoint / subscriber  
-> source ISP NBSR Name Node and Source Edge  
-> authenticated operator tunnel/federation path  
-> destination ISP or cloud NBSR Edge  
-> outbound Origin Connector  
-> private service
```

The source operator can:

- bind traffic to a subscriber/session;
- block spoofed traffic;
- rate-limit by subscriber, device, name, route, tunnel, or service class;
- revoke a compromised client near the source;
- prevent one abusive device from consuming the entire tunnel pool.

The destination operator independently:

- validates source operator and route grant;
- enforces destination quotas and policy;
- rejects untrusted or stale routes;
- protects the origin from raw Internet scans;
- keeps the origin non-public.

NBSR does not create bandwidth and does not eliminate conventional filtering for raw non-NBSR traffic. Its benefit is authenticated admission and earlier discard.

17. Privacy, observability, and logging

Default telemetry SHOULD contain:

- opaque route ID;
- operator and edge IDs;
- profile and protocol version;
- result/error code;
- latency and byte counters;
- policy hash;
- record sequence;
- revocation generation.

Default telemetry SHOULD NOT contain:

- payload;
- route grants or proofs;
- session keys or tokens;
- raw client private identity;
- origin IP;
- long-retention raw service names;
- a reconstructable client-to-origin map.

Raw names may be enabled for controlled troubleshooting with role-based access, short retention, redaction, and audit.

18. Failure semantics

Failure	Required behavior
Name Node unavailable	Legacy names may use configured resilient recursion; new secure-only routes fail closed; active tunnels continue within lease
Route control unavailable	No new unauthorized route; cached service records alone cannot create authorization
Source edge unavailable	Retry another authenticated source edge if route context permits
Destination edge unavailable	Select another signed member of the destination edge set
Origin connector unavailable	Return stable origin-unavailable error without revealing origin location
Revocation distributor partitioned	Stop new high-risk routes when revocation freshness exceeds policy; active routes follow bounded fail-safe policy
Synthetic pool exhausted	Fail explicitly; never return an origin address
Replay state full	Fail closed before upstream connection
Stale or rolled-back record	Reject and preserve the latest accepted sequence
Unsupported protocol/profile	Return explicit unsupported error; no silent downgrade
Tunnel process crash	Legacy name service remains available
Name process crash	Existing tunnels and streams remain operational until lease policy requires revalidation

19. Plug-and-play deployment model

19.1 One package, separated services

The operator-facing package is:

```

NBSR Node
├── nbsr-named      DNS-compatible resolver/authoritative service
├── nbsr-route-controller RouteIntent, grants, policy, and revocation client
├── nbsr-source-edge Synthetic/shared-edge capture and tunnel client
├── nbsr-tunnel-gateway QUIC data plane
├── nbsr-origin-connector Optional destination-side connector
└── nbsr-admin      Migration, health, backup, rollback, and diagnostics

```

Processes use separate service accounts, keys, resource limits, and health checks.

19.2 Upgrade sequence

1. Inventory current DNS zones, recursion, forwarders, DNSSEC, DoH/DoT, DHCP, routes, VPN ranges, and firewall policy.
2. Install NBSR Node in monitor-only mode.
3. Import or forward legacy DNS configuration.

4. Verify legacy name parity against the old resolver.
5. Select collision-free synthetic IPv4/IPv6 ranges.
6. Route those ranges only to the Source NBSR Edge.
7. Enable one test NBSR service record with a private origin connector.
8. Test ordinary clients without an NBSR agent.
9. Enable controlled dual-published services.
10. Enable secure-only mode only after compatibility and rollback tests pass.

19.3 Rollback

The package MUST record only state it owns and provide:

- export of pre-change DNS configuration;
- route and firewall ownership journal;
- exact service/unit changes;
- one-command disable of NBSR record handling;
- removal of synthetic routes added by NBSR only;
- restoration of the prior resolver binding;
- preservation of unrelated VPN, container, WSL, Hyper-V, and administrator routes;
- validation that legacy DNS and Internet access are restored.

Rollback MUST NOT delete pre-existing addresses, routes, DNS zones, certificates, or firewall rules.

20. Current prototype baseline

The hardened July 2026 prototype demonstrates:

- name-route request accepting a hostname;
- synthetic IPv4/IPv6 handles;
- signed 60-second name binding;
- Ed25519 client proof of possession;
- replay rejection in bounded process-local state;
- opaque HTTP and HTTPS TCP relay;
- original HTTPS hostname/SNI and certificate validation;
- server-side-only origin resolution;
- hidden deterministic origin with no host-published port;
- source-visible state that omits the origin address;
- enterprise OPA authorization as a separate optional profile;
- bounded admission, synthetic allocation, and replay caches;
- destination policy blocking non-global targets unless explicitly trusted;
- Docker Compose and Kind reference deployments;

- documented result of 180 passing tests, 1 skipped, Ruff clean, and 5/5 OPA tests on 2026-07-25.

It does not yet demonstrate:

- native NBSR registry/ownership;
- NBSR as a full DNS-compatible resolver;
- QUIC NBSR tunnel profile;
- multiplexed streams;
- route lease renewal;
- live key rotation;
- live migration/resumption;
- distributed replay or revocation;
- HA or multi-zone operation;
- two-operator ISP enforcement;
- federation;
- independent interoperable implementation;
- signed Windows Filtering Platform driver.

The current code is a valuable test oracle and component source. It is not the final wire protocol.

21. Implementation roadmap

Phase	Primary result	Exit criterion
0. V3 lock	Approve this document and terminology	No new architecture contradicts the dual-plane resolver-first model
1. Protocol package	Schemas, state machines, message registry, test vectors	Two in-repo test peers encode/decode identical bytes and reject malformed vectors
2. NBSR Name Node	DNS-compatible legacy resolution plus signed local NBSR records and RouteIntent	Legacy parity tests pass; NBSR name returns only handle state
3. Edge tunnel vertical slice	Source edge, QUIC tunnel, destination edge, outbound connector	Normal client behind source edge reaches private HTTPS origin without learning origin IP
4. Generic TCP and lifecycle	Multiplexing, arbitrary TCP, lease renewal, drain, revoke, key update	Large transfer survives renewal and key update; revoked route blocks new streams
5. Gateway packaging	Linux/OpenWrt and enterprise gateway package	A clean network upgrades and rolls back with no endpoint agent
6. Regional HA	Replicated control, distributed replay/revocation, multi-zone gateways	Zone loss stays within SLO and does not authorize stale routes
7. Two-operator ISP lab	Independent source and destination admission	One abusive subscriber is isolated without affecting others; origin remains unscannable
8. Federation	Signed ownership, delegation, transparency, trust rotation	Cross-operator route survives safe rotation and rejects rollback/split-brain
9. Independent implementation	Second implementation in another language	Both implementations pass the same conformance suite and packet vectors
10. Standardization/pilots	Public draft, security review, vendor/ISP pilots	Interoperability event and documented deployment experience

22. Codex execution rules

Codex MUST follow these rules for every phase:

1. Read this document, the V2 conformance matrix, security hardening report, threat model, current README, and relevant tests before editing.
2. Work on one approved work package only.
3. Write or update a design spec before implementation when behavior changes.
4. Write failing tests or conformance vectors before production code.
5. Preserve the existing enterprise demo unless the approved work package explicitly migrates it.
6. Keep protocol core independent from Kubernetes, Envoy, OPA, cloud SDKs, and operating-system adapters.
7. Keep deterministic tests independent of public DNS and the public Internet.
8. Never expose an origin address in client responses, fixtures, logs, or compatibility state for an NBSR-enabled secure route.
9. Use proof-of-possession grants; do not introduce reusable bearer tickets into the ISP/core profile.
10. Fail closed on ambiguity, stale state, unsupported profiles, replay-cache exhaustion, and signature errors.
11. Separate keys and trust domains by purpose.
12. Do not claim production readiness, full protocol conformance, anonymity, DDoS elimination, or global federation before the exit criteria exist.
13. Run focused tests after each change and the full relevant suite before completion.
14. Report exact commands, observed results, skipped tests, unverified dependencies, and residual risks.
15. Stop at security, architecture, or scope gates and request a human decision rather than guessing.

23. Codex work packages

WP0 - Repository alignment and evidence preservation

Goal: Make V3 the authoritative direction without deleting prototype evidence.

Deliverables:

- place this document under `docs/architecture/`;
- retain V2 and the feasibility study under a clearly labeled historical/supporting directory;
- add `docs/protocol/terminology.md`;
- add `docs/protocol/status.md` separating implemented, partial, planned, and normative behavior;
- update README language that still says DNS is always separate from NBSR;
- preserve all existing tests and demos.

Acceptance:

- no code behavior changes;
- no existing test removal;
- all links resolve;
- no document claims that the prototype already implements Core v0.1.

WP1 - Protocol data model and deterministic vectors

Goal: Freeze byte-level Core v0.1 structures before network implementation.

Deliverables:

- protocol version and message code registry;
- deterministic CBOR encoder/decoder;
- COSE Sign1 helper with explicit Ed25519 allowlist;
- service record, RouteIntent, Route Grant, revocation, and error schemas;
- state-machine types;
- golden valid and invalid vectors;
- fuzz/property tests for length, type, unknown fields, duplicate keys, non-canonical encodings, and signature failures.

Acceptance:

- canonical structures encode to stable bytes;
- decoders reject indefinite or non-canonical forms where prohibited;
- unknown critical fields fail closed;
- signature algorithm confusion is impossible;
- vectors contain no secrets or origin addresses.

WP2 - NBSR Name Node

Goal: Make NBSR the configured resolver for both legacy and NBSR names.

Deliverables:

- UDP/TCP DNS-compatible listener for lab use;
- legacy recursion/forwarding adapter with deterministic fake upstream in tests;
- signed local NBSR registry;
- classification into legacy/NBSR/negative;
- RouteIntent store;
- Resolution Context ID adapter;
- synthetic handle allocator with collision and capacity controls;
- stable metrics and privacy-safe logs.

Acceptance:

- existing DNS fixtures return byte-equivalent or semantically equivalent expected records;
- NBSR names never return origin A/AAAA data;
- Name Node stays available when tunnel service is stopped;
- an NBSR record signature, sequence, or revocation failure returns a secure error;
- pool exhaustion never falls back to origin IP.

WP3 - QUIC edge and origin connector

Goal: Build the first native secure route.

Deliverables:

- `nbsr-quic-1` handshake;
- source and destination mutual authentication;
- one control stream;
- Route Grant verification and PoP;
- source/destination admission;
- outbound origin connector;
- opaque HTTPS stream forwarding;
- explicit accepted/rejected response before payload.

Acceptance:

- standard browser/curl behind the source edge reaches a private HTTPS test origin;
- the endpoint validates the original origin certificate;
- the origin IP is absent from client-visible state;
- raw scanning cannot reach the origin;
- tampering, replay, wrong name, wrong edge, wrong port, wrong key, expiry, and algorithm downgrade are rejected;
- stopping the Name Node does not interrupt the active accepted stream within lease policy.

WP4 - Multiplexing and lifecycle

Goal: Turn one secure relay into a reusable protocol tunnel.

Deliverables:

- multiple independent TCP streams;
- per-stream admission;
- route lease renewal;
- idle and maximum-lifetime timers;
- drain and completion grace;

- immediate revoke mode;
- key update without stream interruption;
- bounded backpressure and quotas.

Acceptance:

- a large transfer survives renewal and key update;
- a denied renewal blocks new streams;
- immediate revoke terminates according to policy;
- one blocked stream does not deadlock unrelated streams;
- resource limits fail predictably.

WP5 - Gateway packaging and no-agent lab

Goal: Prove the "DNS upgrade" operational model.

Deliverables:

- Linux service package or containers;
- OpenWrt-compatible packaging design;
- DNS import/forwarder configuration;
- synthetic route installer with ownership journal;
- firewall policy;
- install, verify, backup, rollback, and uninstall commands;
- health and parity checks.

Acceptance:

- a clean client device requires no NBSR installation;
- legacy websites continue to work;
- one NBSR test service uses the secure route;
- uninstall restores previous behavior;
- unrelated DNS, routes, VPNs, firewall rules, and certificates are untouched.

WP6 - Distributed security state and HA

Goal: Remove single-process security assumptions.

Deliverables:

- partition-safe replay strategy;
- replicated RouteIntent and lease state;
- bounded revocation propagation;
- multi-zone source/destination edges;
- stale-state policy;

- chaos tests;
- KMS/HSM integration interface;
- signed configuration and release verification.

Acceptance:

- loss of one zone stays within SLO;
- a replay sent to another replica is rejected;
- stale revocation state blocks according to policy;
- active streams survive permitted control-plane outages;
- new routes do not bypass failed authorization.

WP7 - Two-operator ISP lab

Goal: Demonstrate the core value across independent administrative domains.

Deliverables:

- ISP-A source edge and ISP-B destination edge;
- separate keys, policies, logs, and rate limits;
- simulated subscriber contexts;
- cross-operator route trust;
- abuse and overload tests.

Acceptance:

- abusive subscriber is limited at ISP-A;
- other subscribers remain unaffected;
- ISP-B performs independent admission;
- spoofed traffic is rejected;
- origin is not discoverable by raw scan;
- source operator does not learn origin IP;
- destination operator does not receive raw endpoint IP;
- per-client, per-name, per-route, and per-tunnel limits work.

WP8 - Federation, second implementation, and standards

Goal: Cross the boundary from one software project to a protocol.

Deliverables:

- signed ownership/delegation;
- transparency log;
- rollback and split-brain handling;
- trust-bundle rotation;

- independent implementation in a different language;
- packet captures;
- public conformance suite;
- Internet-Draft.

Acceptance:

- two implementations exchange the same route and stream protocol;
- both pass identical valid and invalid vectors;
- key compromise and rotation drills are documented;
- federation rejects stale ownership and malicious peer behavior.

24. Conformance test catalog

24.1 Name-plane tests

- legacy A, AAAA, CNAME, MX, TXT, SRV, SVCB, HTTPS, DNSSEC pass-through/serving as configured;
- signed NBSR record accepted;
- unknown owner rejected;
- stale sequence rejected;
- expired record rejected;
- revoked key rejected;
- secure-only name never returns origin address;
- negative caching does not become authorization;
- resolver/tunnel process isolation;
- cache and pool exhaustion.

24.2 Route-security tests

- valid grant and PoP accepted;
- tampered grant rejected;
- wrong name/service/transport/port rejected;
- wrong source/destination edge rejected;
- wrong session key rejected;
- replay rejected across replicas;
- expired/not-yet-valid grant rejected;
- unsupported algorithm/profile rejected;
- downgrade attempt rejected;
- stale revocation generation rejected.

24.3 Data-plane tests

- opaque HTTP;
- end-to-end HTTPS with original certificate and SNI;
- SSH or deterministic arbitrary TCP;
- multiple concurrent streams;
- backpressure;
- stream reset isolation;
- destination failover;
- connector drain;
- large transfer during renewal and key update.

24.4 Privacy and concealment tests

- origin absent from name response;
- origin absent from client route/cache state;
- origin absent from normal client logs;
- client IP absent at private origin;
- raw scan cannot reach origin;
- payload not terminated in relay-only profile;
- telemetry cannot reconstruct client-to-origin mapping from default fields.

24.5 Compatibility and rollback tests

- non-NBSR name parity;
- dual-published service behavior;
- secure-only fail-closed behavior;
- endpoint with no agent;
- resolver upgrade and rollback;
- route/firewall ownership journal;
- existing VPN/container/WSL/Hyper-V routes preserved.

25. Error registry

Code	Meaning
NBSR_E_NAME_INVALID	Name is syntactically or canonically invalid
NBSR_E_NAME_NOT_FOUND	No legacy or NBSR record exists
NBSR_E_RECORD_UNTRUSTED	Ownership or signature cannot be validated
NBSR_E_RECORD_STALE	Record sequence is older than accepted state

Code	Meaning
NBSR_E_RECORD_REVOKED	Record or signing key is revoked
NBSR_E_CONTEXT_REQUIRED	Trustworthy Resolution Context ID is unavailable
NBSR_E_HANDLE_EXHAUSTED	Synthetic handle capacity is unavailable
NBSR_E_ROUTE_DENIED	Route policy denied admission
NBSR_E_GRANT_INVALID	Route Grant is malformed or cryptographically invalid
NBSR_E_GRANT_EXPIRED	Route Grant is outside its validity interval
NBSR_E_PROOF_INVALID	Proof of possession failed
NBSR_E_REPLAY	Nonce or admission material was reused
NBSR_E_PROFILE_UNSUPPORTED	Requested protocol/transport profile is unsupported
NBSR_E_DOWNGRADE	Negotiation attempted an unauthorized weaker profile
NBSR_E_EDGE_UNAVAILABLE	No authorized edge is healthy
NBSR_E_ORIGIN_UNAVAILABLE	Registered connector/service is unavailable
NBSR_E_REVOKED	Active route was revoked
NBSR_E_OVER_CAPACITY	A bounded control or data-plane resource is exhausted
NBSR_E_INTERNAL	Stable non-sensitive internal failure

Error messages **MUST NOT** reveal origin addresses, keys, tokens, trust internals, or policy details.

26. Ready-to-use Codex directive

The following directive is the starting instruction for the first implementation cycle:

You are implementing Name-Based Secure Routing Protocol Vision V3 in the current NBSR repository.

Authoritative direction:

- NBSR is the configured name resolver for both legacy and NBSR-enabled names.
- Legacy names keep DNS-compatible behavior.
- NBSR-enabled names resolve to secure route state, never an origin IP.
- The deployable product has two isolated planes: Name/Resolution and Secure Route/Tunnel.
- A Source NBSR Edge acts as the protocol client for legacy devices behind an upgraded router, enterprise gateway, or ISP. A per-device agent is not required for the first lab.
- The existing IP Internet remains the underlay between NBSR edges.

Scope for this cycle: WP0 and WP1 only.

Before editing:

1. Read NBSR Protocol Vision V3, the V2 conformance matrix, the security hardening report, the threat model, README, and current name-routing tests.
2. Inspect the repository and report the current branch, dirty files, test commands, and any conflict with V3.
3. Write a concrete implementation plan and stop for review before production code changes.

WP0:

- Add V3 under docs/architecture/.
- Preserve V2 and the feasibility study as historical/supporting documents.
- Add terminology and implementation-status documents.
- Correct README wording that treats DNS as permanently separate from NBSR.
- Do not change behavior or remove tests.

WP1:

- Add deterministic CBOR Core v0.1 message types and stable numeric codes.
- Add COSE Sign1 Ed25519 helpers with an explicit algorithm allowlist.
- Define service record, RouteIntent, Route Grant, revocation, and error schemas with owner/operator/sequence/revocation fields.
- Add state-machine types and valid/invalid golden vectors.
- Test canonical encoding, malformed lengths/types, duplicate keys, non-canonical encodings, unknown critical fields, signature failure, algorithm confusion, expiry, stale sequence, and rollback.

Non-negotiable constraints:

- Test first.
- Keep protocol core independent of Kubernetes, Envoy, OPA, cloud SDKs, and OS adapters.
- Use deterministic local fixtures; do not depend on public DNS or Internet services.
- Never place origin addresses in NBSR-enabled client responses, fixtures, or logs.
- Do not introduce bearer authorization into the ISP/core profile.
- Fail closed on ambiguity, replay, capacity exhaustion, stale state, unsupported profiles, and cryptographic errors.
- Preserve the enterprise demo.
- Do not claim production readiness or full V3 conformance.

Verification before completion:

- Run focused WP0/WP1 tests.
- Run the existing full Python and policy suites where dependencies are

available.

- Run lint/static checks.
- Report exact commands, observed results, skipped checks, and residual gaps.
- Stop after WP1. Do not begin the Name Node or QUIC implementation without a separate approved plan.

27. Human decision gates

The following decisions require explicit human approval before the relevant work package:

1. Final CBOR/COSE library and cross-language compatibility policy.
2. Operator identity format and trust-anchor model.
3. Production synthetic IPv4/IPv6 allocation rules.
4. Exact Linux/OpenWrt traffic-capture mechanism.
5. Immediate revocation versus completion-grace policy.
6. Distributed replay consistency model during partitions.
7. Federation ownership and transparency governance.
8. Raw-name logging and retention policy.
9. Independent second implementation language.
10. Public standardization venue and licensing/patent declarations.

28. Definition of protocol success

NBSR becomes a protocol, rather than only software, when:

- the wire structures and state machines are public and versioned;
- security and privacy behavior are normative;
- packet captures and deterministic vectors exist;
- a conformance suite exercises success and failure behavior;
- at least two independent implementations communicate;
- both implementations pass the same suite;
- deployment experience exists in a no-agent upgraded network and a two-operator lab;
- known limitations and residual risks are published;
- the project enters a credible standards process.

Until then, the correct description is:

NBSR is a pre-standardization protocol project with a hardened name-routing prototype and an explicit path to interoperable implementation.

Appendix A - Normative language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** are to be interpreted as described by RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

Appendix B - Standards baseline

Core design references:

- RFC 1035 - Domain names: implementation and specification
- RFC 8446 - TLS 1.3
- RFC 9000 - QUIC: A UDP-Based Multiplexed and Secure Transport
- RFC 9001 - Using TLS to Secure QUIC
- RFC 8949 - Concise Binary Object Representation (CBOR)
- RFC 9052 - CBOR Object Signing and Encryption (COSE) structures
- RFC 8032 - Edwards-Curve Digital Signature Algorithm (EdDSA)
- RFC 2119 and RFC 8174 - Normative requirement language
- RFC 5890/RFC 5891 - Internationalized domain-name definitions and processing
- RFC 9460 - Service Binding and HTTPS DNS resource records, used only for migration/compatibility where appropriate

Implementation and research sources:

- *Name-Based Secure Routing* feasibility study, 19 pages
- *NBSR Protocol Vision and Non-Negotiable Design Principles*, Version 2.0
- Hardened prototype README
- NBSR Protocol Vision v2 conformance matrix
- NBSR security hardening report
- NBSR threat model
- Current name-service, relay, synthetic-address, Windows-agent, policy, and integration tests

Appendix C - Final anti-drift checklist

Every major design, feature, pull request, or Codex task must answer:

1. Does the upgraded network use NBSR as the resolver rather than placing NBSR only after DNS?
2. Do legacy names still work without an Internet-wide flag day?
3. Does an NBSR-enabled name become route state rather than an origin IP?
4. Can a normal endpoint work without an NBSR agent behind an upgraded source edge?
5. Are Name/Resolution and Secure Route/Tunnel isolated?
6. Is the source gateway on a real traffic path for compatibility handles?

7. Is the route authenticated, encrypted, integrity-protected, name-bound, and proof-of-possession-bound?
8. Are source and destination admission independent?
9. Can the origin remain private and unscannable?
10. Are cache TTL, handle lifetime, route lease, and tunnel lifetime distinct?
11. Is automatic downgrade to raw origin resolution impossible for secure-only names?
12. Are replay, rollback, stale delegation, revocation, capacity, and partition behavior defined?
13. Does the protocol work outside Kubernetes?
14. Are enterprise IAM and inspection optional extensions?
15. Does the change preserve end-to-end application TLS where the relay-only profile is used?
16. Does the test prove behavior without relying on public Internet services?
17. Is rollback safe and ownership-scoped?
18. Does the implementation claim only what evidence proves?